



Security Assessment Report

Static Application Security Testing & Code Assessment

PROJECT

SecureCode Test Project

SCAN NAME

Python Security Test

UPLOADED FILE

vulnerable_app.py

ASSESSMENT TYPE

Initial Assessment

LANGUAGE

python

ASSESSMENT DATE

21 September 2026

PREPARED BY

SecureCode Auditor Platform

CONFIDENTIAL

Document Control

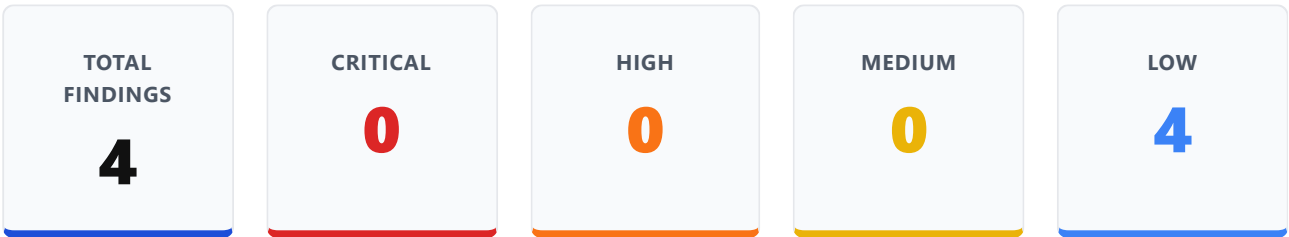
Document Title	Security Assessment Report
Application	SecureCode Auditor
Project	SecureCode Test Project
Scan Name	Python Security Test
Uploaded File	vulnerable_app.py
Assessment Type	INITIAL
Scan ID	6ab101efd519d49e7f24aab2
Programming Language	python
Scanner	SecureCode Auditor Platform
Assessment Date	21/9/2026, 3:37:43 pm
Report Generated	5/10/2026, 9:41:02 pm
Report Version	1.0
Classification	CONFIDENTIAL

Table of Contents

- 1. Executive Summary
- 2. Assessment Overview
- 3. Assessment Methodology
- 4. Scope
- 5. Risk Summary
- 6. Findings Summary
- 7. Detailed Findings
 - 7.4 Low Findings
- 8. Remediation & Recommendations
- 9. Conclusion
- 10. References
- 11. Appendix

1. Executive Summary

This security assessment was automatically performed using the SecureCode Auditor platform to identify, classify, and provide remediation guidance for security weaknesses within the assessed source code.



Overall Security Risk: LOW

The assessed codebase contains only low severity or informational findings. Security posture is relatively strong, but defense-in-depth improvements can be made.

2. Assessment Overview

Project	SecureCode Test Project
Scan Name	Python Security Test
Uploaded File	vulnerable_app.py
Language	python
Scanner	SecureCode Auditor Platform
Scan Type	INITIAL
Assessment Date	21/9/2026, 3:37:43 pm

3. Assessment Methodology

The security assessment follows a rigorous lifecycle leveraging both static analysis and AI-assisted contextual validation:

- 1. **Static Analysis:** The source code is scanned using industry-standard static application security testing (SAST) engines to identify vulnerable patterns.
- 2. **Vulnerability Identification:** Findings are mapped to standard frameworks (CWE, OWASP).
- 3. **AI-Assisted Analysis:** Validated findings are analyzed by an advanced AI model to determine context, root cause, and precise secure code remediation.
- 4. **Manual Security Review:** Human security analysts review critical findings to validate True/False positives and assess business risk.
- 5. **Reporting:** A comprehensive assessment report is compiled detailing all evidence and actionable remediation steps.

4. Scope

The scope of this assessment includes the source code provided to the SecureCode Auditor platform during the initiation of the scan.

Project Scope	SecureCode Test Project
Target Language	python
Environment	Static Source Code

5. Risk Summary



6. Findings Summary

ID	Finding	Severity	Status	CWE	Review
F-001	blacklist	LOW	OPEN	N/A	N/A
F-002	hardcoded_password_string	LOW	OPEN	N/A	N/A
F-003	start_process_with_partial_path	LOW	OPEN	N/A	N/A
F-004	subprocess_popen_with_shell_equals_true	LOW	OPEN	N/A	N/A

7. Detailed Findings

F-001

blacklist LOW

Status	OPEN	Scanner	N/A
Rule ID	N/A	File	source.py
CWE	N/A	Line	1
OWASP	N/A		

Description

Consider possible security implications associated with the subprocess module.

Security Impact

Confidence: HIGH

AI-ASSISTED SECURITY ANALYSIS

Summary

The code imports or uses the `subprocess` module, which can lead to security vulnerabilities if not handled correctly, such as command injection or arbitrary code execution.

Why It Is Vulnerable

The `subprocess` module in Python is used to spawn new processes, pipe their standard input/output/error, and get their return codes. If user-controlled input is passed to functions like `subprocess.call`, `subprocess.run`, or `subprocess.Popen` with `shell=True`, it can lead to command injection. Even without `shell=True`, improper handling of arguments can lead to security issues. The boundary affected is the interface between the Python application and the operating system's shell or process execution environment.

Root Cause

Use of the `subprocess` module without proper input validation, sanitization, or secure configuration (e.g., avoiding `shell=True` and using argument lists instead of strings).

Recommendation

1. Avoid using `shell=True` in subprocess functions. 2. Pass arguments as a list rather than a string to prevent shell interpretation. 3. Validate and sanitize all user input before passing it to subprocess functions. 4. Use `subprocess.run` with `check=True` to handle errors properly. 5. Consider using higher-level abstractions or libraries that provide safer process execution if possible.

Secure Code Example

```
import subprocess

# Insecure: Using shell=True with user input
# subprocess.call(f"echo {user_input}", shell=True)

# Secure: Using argument list without shell=True
user_input = "hello"
subprocess.run(["echo", user_input], check=True)
```

REFERENCES

[R1] Bandit Documentation: B404 - Consider possible security implications associated with the subprocess module

External Reference

[Bandit Documentation: B404 - Consider possible security implications associated with the subprocess module](#)

[R2] Python Official Documentation: subprocess - Subprocess management

External Reference

[Python Official Documentation: subprocess - Subprocess management](#)

F-002

hardcoded_password_string LOW

Status	OPEN	Scanner	N/A
Rule ID	N/A	File	source.py
CWE	N/A	Line	2
OWASP	N/A		

Description

Possible hardcoded password: 'Admin123'

Security Impact

Confidence: MEDIUM

AI-ASSISTED SECURITY ANALYSIS

AI analysis was not generated for this finding.

F-003

start_process_with_partial_path LOW

Status	OPEN	Scanner	N/A
Rule ID	N/A	File	source.py
CWE	N/A	Line	3
OWASP	N/A		

Description

Starting a process with a partial executable path

Security Impact

Confidence: HIGH

AI-ASSISTED SECURITY ANALYSIS

AI analysis was not generated for this finding.

F-004

subprocess_popen_with_shell_equals_true LOW

Status	OPEN	Scanner	N/A
Rule ID	N/A	File	source.py
CWE	N/A	Line	3
OWASP	N/A		

Description

subprocess call with shell=True seems safe, but may be changed in the future, consider rewriting without shell

Security Impact

Confidence: HIGH

AI-ASSISTED SECURITY ANALYSIS

AI analysis was not generated for this finding.

8. Remediation & Recommendations

ID	Severity	Recommended Action
F-001	LOW	1. Avoid using `shell=True` in subprocess functions. 2. Pass arguments as a list rather than a string to prevent shell interpretation. 3. Validate and sanitize all user input before passing it to subprocess functions. 4. Use `subprocess.run` with `check=True` to handle errors properly. 5. Consider using higher-level abstractions or libraries that provide safer process execution if possible.
F-002	LOW	Review the code and consult Bandit documentation for B105
F-003	LOW	Review the code and consult Bandit documentation for B607
F-004	LOW	Review the code and consult Bandit documentation for B602

9. Conclusion

The assessment of **SecureCode Test Project** resulted in the identification of **4** vulnerabilities, resulting in an overall risk rating of **LOW**.

The assessed codebase contains only low severity or informational findings. Security posture is relatively strong, but defense-in-depth improvements can be made.

It is strongly recommended that the development team reviews the detailed findings in Section 7 and applies the remediation steps outlined in Section 8. Critical and High severity findings should be prioritized in the immediate sprint to minimize organizational risk. Upon completion of remediation efforts, a retest scan should be performed to validate the integrity of the implemented fixes.

10. References

Reference Traceability

Finding	References
F-001	[R1], [R2]
F-002	No external references available.
F-003	No external references available.
F-004	No external references available.

Master Reference Registry

[R1] Bandit Documentation: B404 - Consider possible security implications associated with the subprocess module

Source: External Reference

[Bandit Documentation: B404 - Consider possible security implications associated with the subprocess module](#)

Used by: **F-001**

[R2] Python Official Documentation: subprocess - Subprocess management

Source: External Reference

[Python Official Documentation: subprocess - Subprocess management](#)

Used by: **F-001**

11. Appendix

A. Scan Metadata

Scan ID	6ab101efd519d49e7f24aab2
Generated On	5/10/2026, 9:41:02 pm
Platform Engine	SecureCode Auditor Core

B. AI Model Information

The following AI models were utilized during the analysis of findings in this report:

- qwen/qwen3.8-27b

Note: AI analysis is strictly limited to static technical context and never introduces fabricated telemetry or false references.